

Customer Journey Map — Script-Controlled ACL

Restrict Record Access Based on Field Value (u_institution_details table)

Persona 1: EEE Branch User

Roles held: bb1 (custom role) | **Goal:** *View student/institution records belonging to the EEE branch*

Journey Stage	Trigger & Intent	Login / Navigation	Record Access Attempt	ACL Script Evaluation	System Outcome	User Emotion
What they do	Wants to check institution / student records for their EEE branch	Logs into ServiceNow as 'EEE User' (User ID: EEEUser)	Opens Institution Details list (u_institution_details.list)	Read ACL fires: gs.hasRole('admin')→false, then gs.hasRole('bb1')→true; Data condition Branch = EEE	List loads showing only the 2 records where Branch = EEE (INS0001007, INS0001004)	Confident — sees exactly the records relevant to their branch, nothing more
System / ACL behind the scenes	No ACL check yet — user only has intent	Standard authentication; no ACL involved	GlideRecord query triggers row-level Read ACL evaluation for every record	Script returns true only when role = bb1 AND Branch = EEE; false for ECE/CSE rows	Non-EEE rows silently filtered out of the list — no error shown, just absent	N/A — user-facing outcome, not a system step
Pain point / risk	May not know which records they're even entitled to see	If bb1 role is missing/removed, list appears completely empty with no explanation	Misconfigured Data Condition (e.g. wrong Branch value) can silently hide valid records		'No records to display' gives no reason — can confuse a legitimate EEE user	Frustration if access is denied without a clear message
Opportunity to improve	Publish a simple 'who can see what' access guide	Add a UI policy / info message explaining branch-based filtering	Add ACL script comments + version control on Data Conditions		Show a friendly 'restricted by branch' message instead of a blank list	Proactive communication reduces support tickets

Persona 2: System Administrator

Roles held: admin (+ bb1, bb2, bb3, bb4 implicitly overridden) | **Goal:** *Full Read / Create / Write / Delete access to all branch records, regardless of value*

Journey Stage	Trigger & Intent	Login / Navigation	Record Access Attempt	ACL Script Evaluation	System Outcome	User Emotion
What they do	Needs to manage ACLs and oversee all institution records (ECE, EEE, CSE)	Logs in as System Administrator; elevates to 'security_admin' to edit ACLs	Opens Institution Details list — sees New, Edit, Delete actions available	Every ACL (Read/Create/Write/Delete) hits gs.hasRole('admin') first → returns true immediately	All 6 records visible (ECE, EEE, CSE); can create, edit and delete any record	In control — full visibility and authority over the data
System / ACL behind the scenes	N/A	Elevate Role dialog grants temporary security_admin privileges for this session	Read ACL script short-circuits on the admin check before the bb1 + Branch condition runs	Write ACL (bb3) and Delete ACL (bb4) have no Data Condition — admin override applies via 'Admin overrides' checkbox	Record list, New button, and Delete action all render since every operation is allowed	N/A
Pain point / risk	Elevated privileges must be remembered to log out of	Elevation expires only at logout — risk if session left open	Easy to accidentally edit/delete records outside one's own branch	'Admin overrides' bypassing all Data Conditions is powerful — misuse has wide blast radius	No branch-based audit trail shown to flag why admin changed a record	Overconfidence can lead to unintended cross-branch edits
Opportunity to improve	Time-bound elevation reminders	Auto-expire elevated role after a fixed idle period	Add confirmation prompts for cross-branch edits/deletes	Log every admin-override action to a dedicated audit table	Show a branch tag/banner on records being edited by admin	Builds accountability without slowing the admin down

Outcome: *Script-controlled ACLs evaluate role + record field value together at the Read/Create/Write/Delete level, so EEE-branch users see only EEE records while admins retain unrestricted access across the Institution Details table.*